PA4 Write-up: A-Frame VR Scene

Saule Amangeldyyeva · Student ID 241487 · 3 November 2026

Variant and setup

Student ID 241487 gives prefix `x1487-`, theme winter steppe, **pulse** motion, **gaze** interaction (1.5 s fuse) and **corridor** layout. The scene uses A-Frame 1.8.0 in Chrome, served by a local server. Every screenshot shows the ID panel at the end of the corridor.

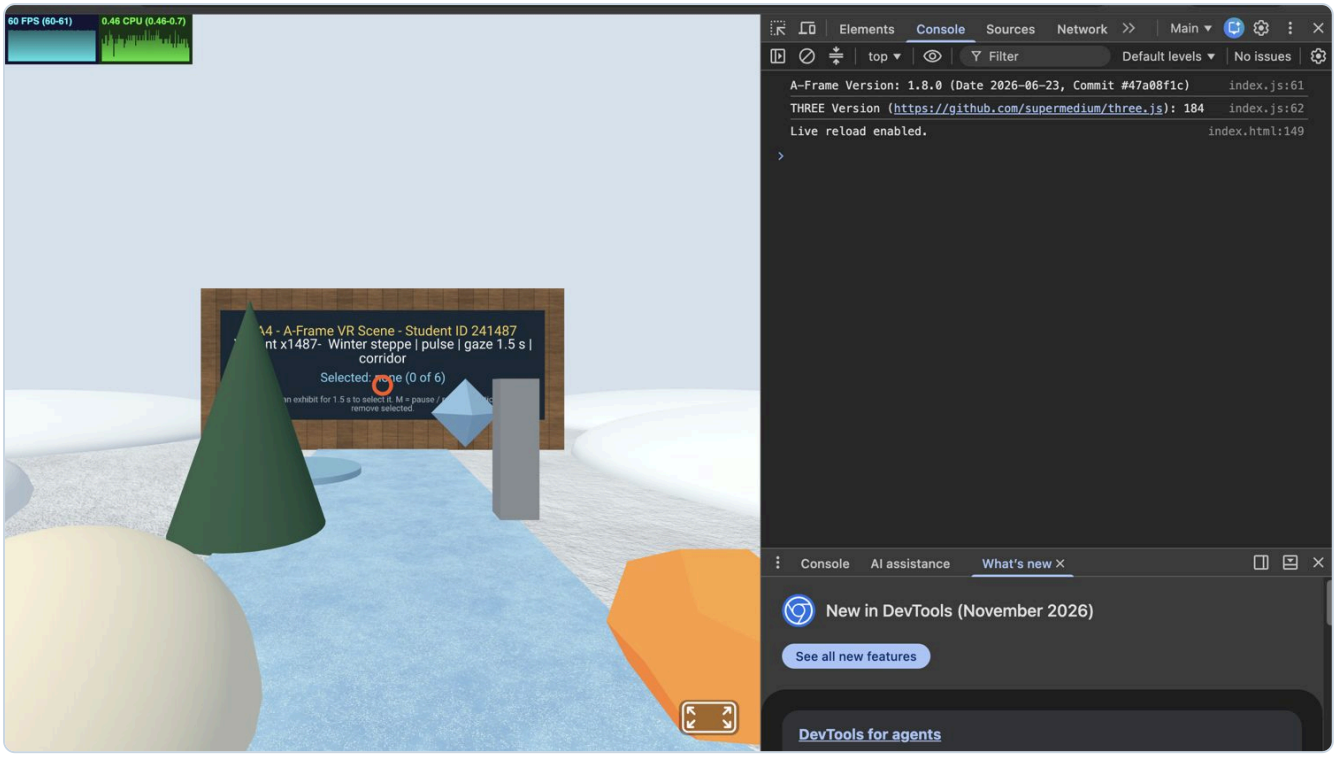


Figure 1. Scene, ID board, clean console.

Enter VR was tested in the Immersive Web Emulator (Meta Quest 3 profile): the session started, the scene kept running at 60 fps and the ID board stayed readable.

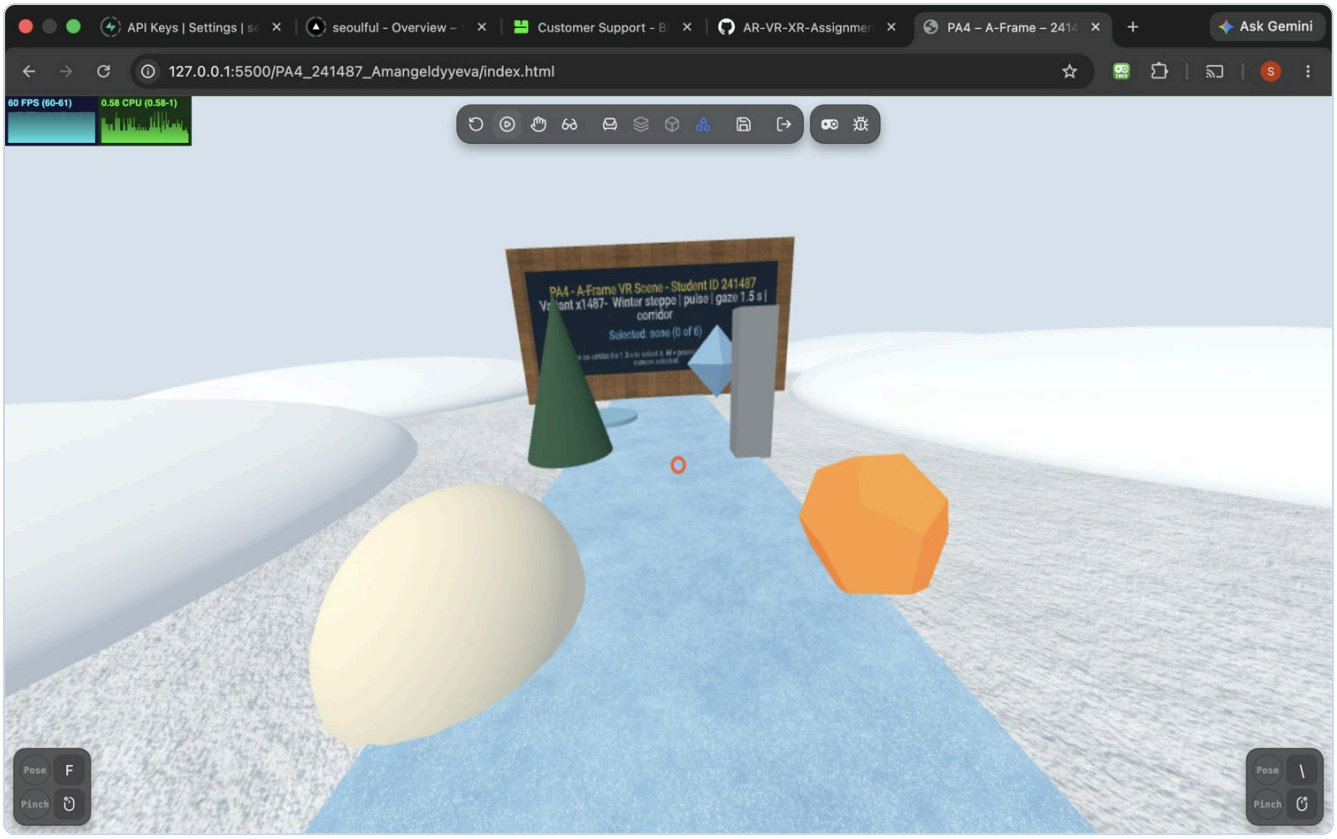


Figure 1b. Enter VR in the emulator.

E1 – Life cycle of x1487-motion

Prediction. `init` runs once, then `update` with an empty `oldData`; each `setAttribute` calls `update` with the old values; `pause()` / `play()` call only those methods; `removeAttribute` pauses and removes the component.

Result. At page load each of the six exhibits logged `init` followed by `update` with `oldData={}` and a different `phase` (0, 1.05, 2.1, 3.15, 4.2, 5.25). Then `setAttribute` gave one `update` whose `oldData` held amplitude 0.15 and whose `data` held 0.4; `removeAttribute` printed `pause` then `remove`; removing the entity printed nothing, and the counter on the board changed to `of 5`.

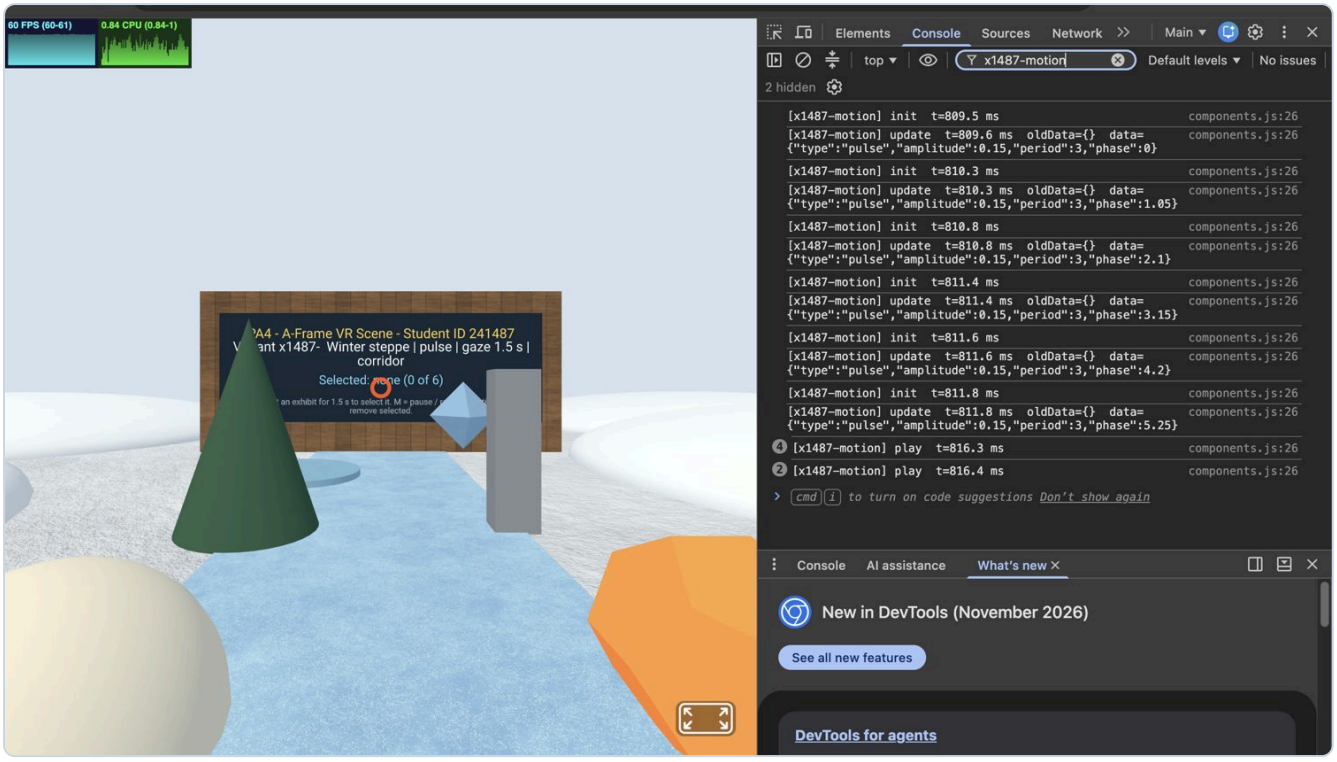


Figure 2. Page-load log.

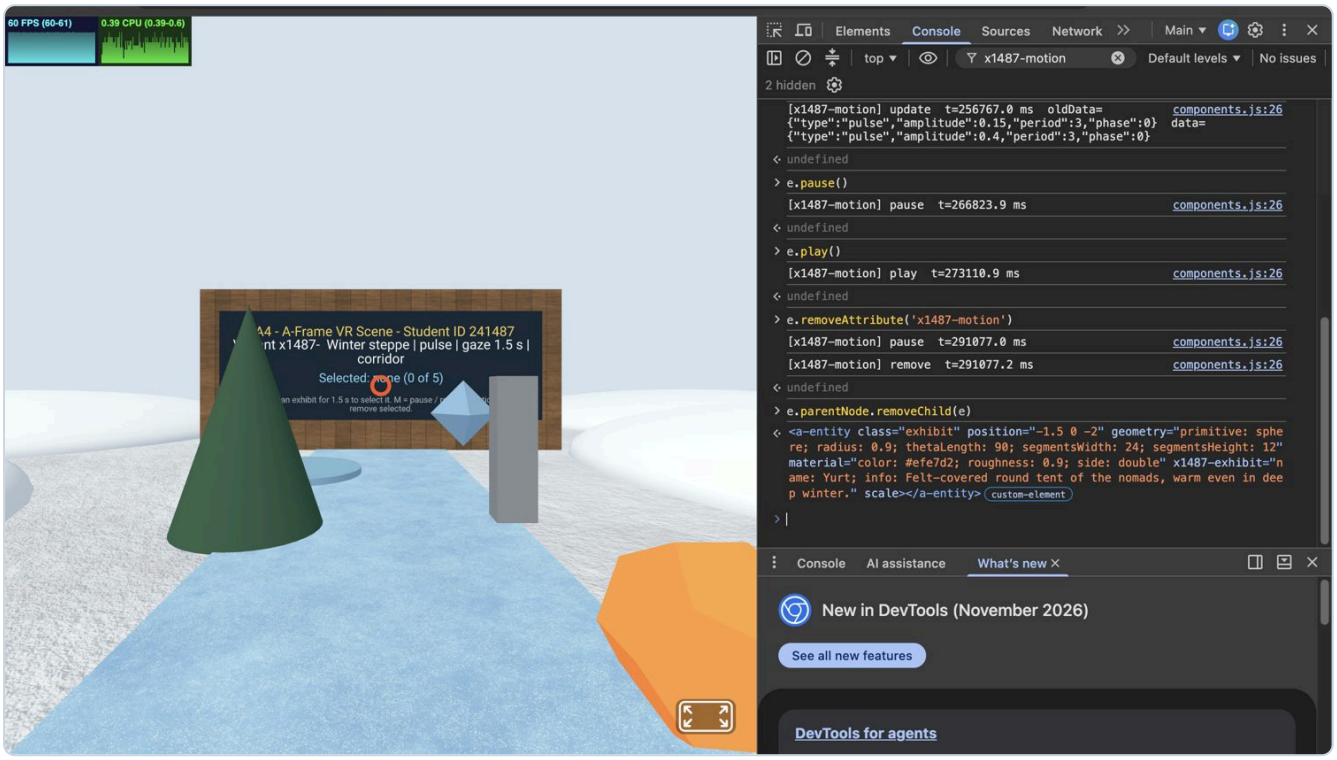


Figure 3. Console sequence and board.

Explanation. A-Frame calls `update` right after `init`, so `oldData` is `{}` on the first call. Later it fires on every data change and `oldData` holds the previous values. Entity `pause()` / `play()` stop and restart `tick`; removal pauses the component, then calls `remove`, where I restore the base pose.

E2 – Checking Ch. 8

(a) Prediction. Changing only `src` re-runs `update`, which rebuilds the texture and material stored in `this.data`.

Result. `this.data` had the keys `src, material_tex, mesh, texture` before and after `setAttribute('texture-loader','src','../textures/ice.jpg')`; only `src` changed. The texture object was replaced: `same texture object?` printed `false`. The old texture is not disposed anywhere in the chapter's code. The page is the original Ch. 8 demo, so it has no ID panel.

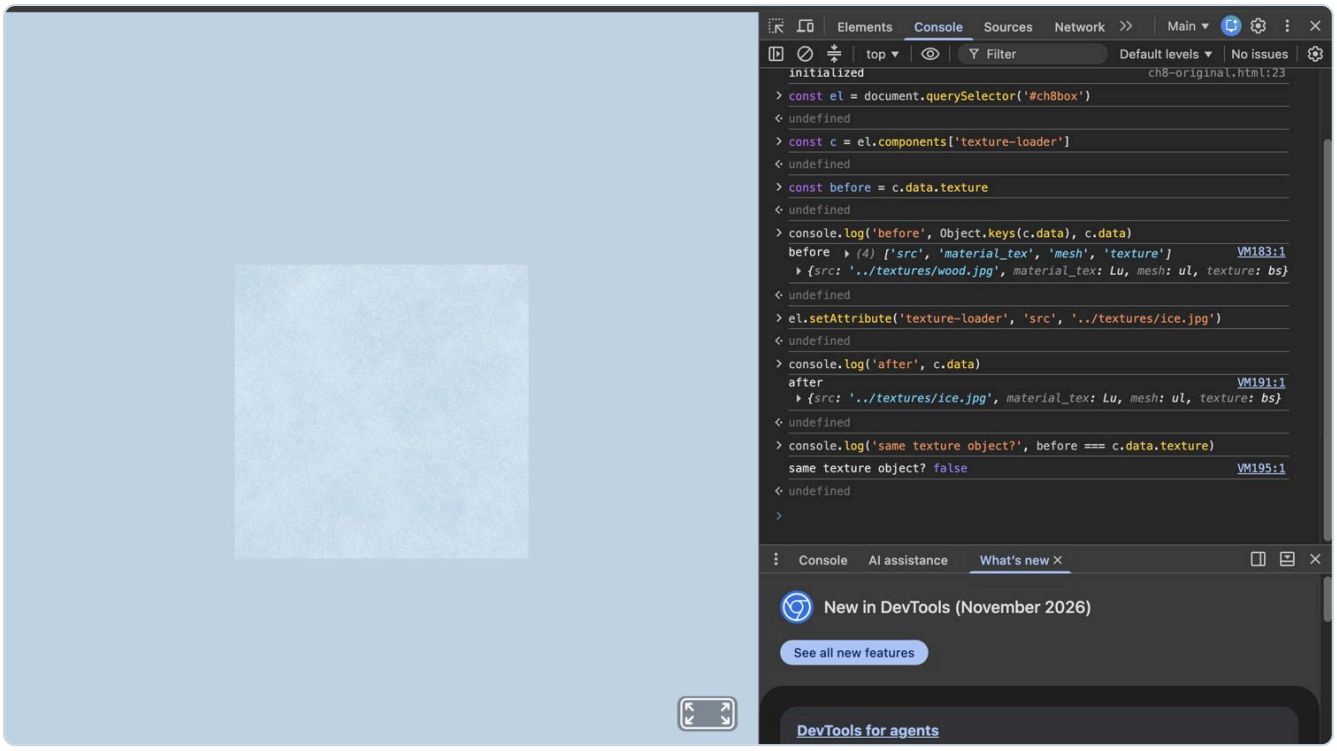


Figure 4. Ch. 8 data before and after.

Explanation. `this.data` is A-Frame's parsed copy of the attribute and is meant for configuration. Runtime objects stored there are recreated on every update and leak GPU memory. Texture, material and mesh belong on `this` (for example `this.texture`), which A-Frame never touches, so they can be reused and disposed in `remove`.

(b) Prediction. `<a-assets>` is not a system.

Result. `Object.keys(AFRAME.scenes[0].systems)` returned 11 systems with no `assets` entry, while `document.querySelector('a-assets')` returned an `<a-assets>` custom HTML element.

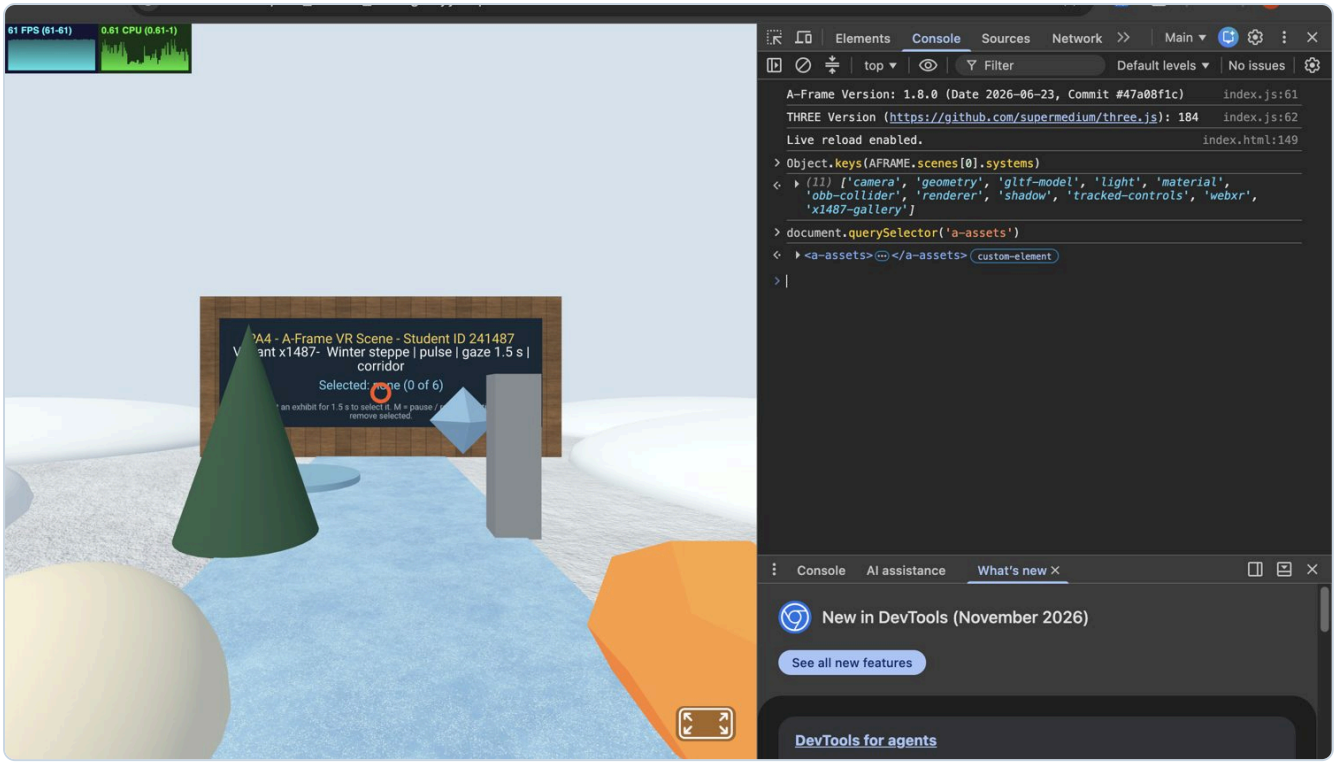


Figure 5. Systems list and a-assets.

Explanation. `AFRAME.registerSystem` creates one singleton per scene with `init` and `tick`, reachable as `sceneEl.systems[name]`, with no DOM element. `<a-assets>` is an HTML element that preloads files and delays the scene's `loaded` event, so the chapter's claim does not hold.

E3 – Texture filtering

Prediction. On a low grazing view of the 16 × 16 snow, the Ch. 8 filters shimmer most, the defaults blur the distance, and mipmaps plus anisotropy look sharpest and calm.

Result. From the same low camera position, the Ch. 8 filters (Figure 6) give the grainiest snow, with sharp dark and bright specks that flicker as the camera moves. The three.js defaults (Figure 7) are smoother. My filters (Figure 8) are smoothest, and the difference from the defaults is small and visible mainly in the far distance. `'maxFilter' in new THREE.Texture()` returned `false`.

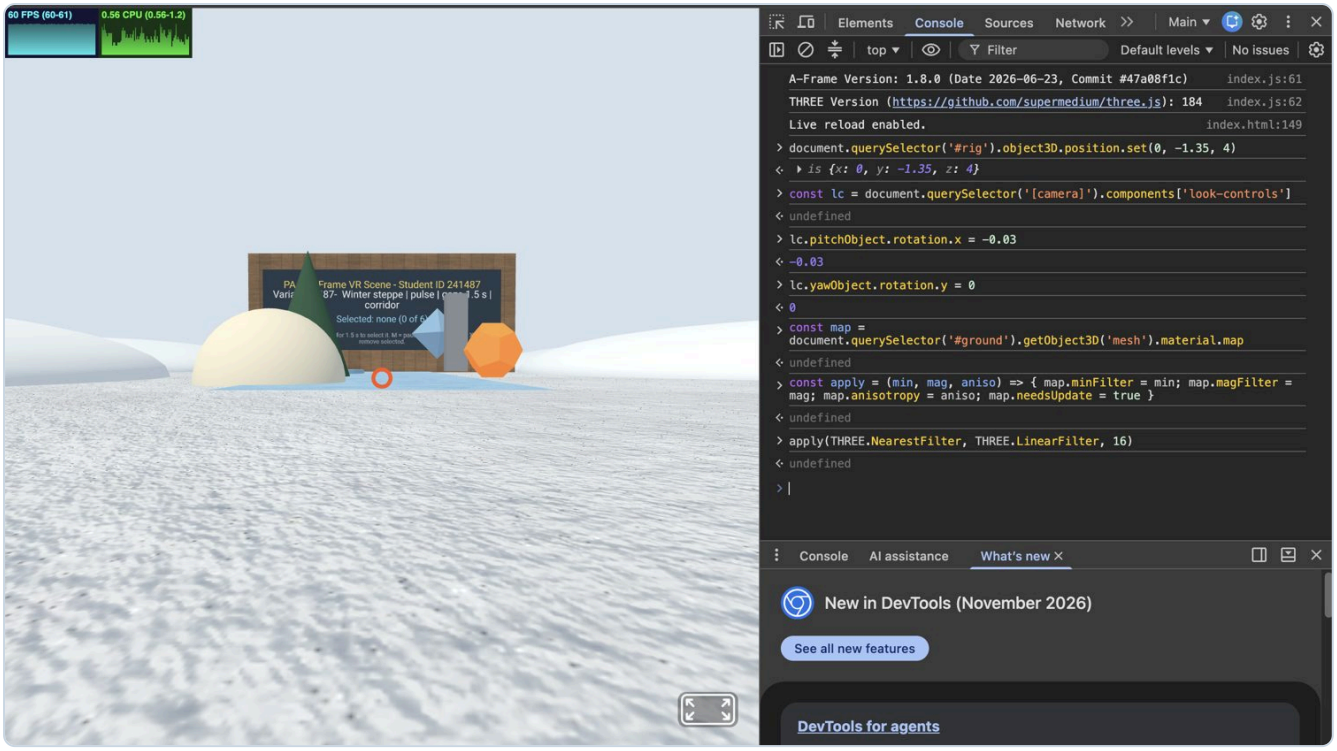


Figure 6. Ch. 8 filters.

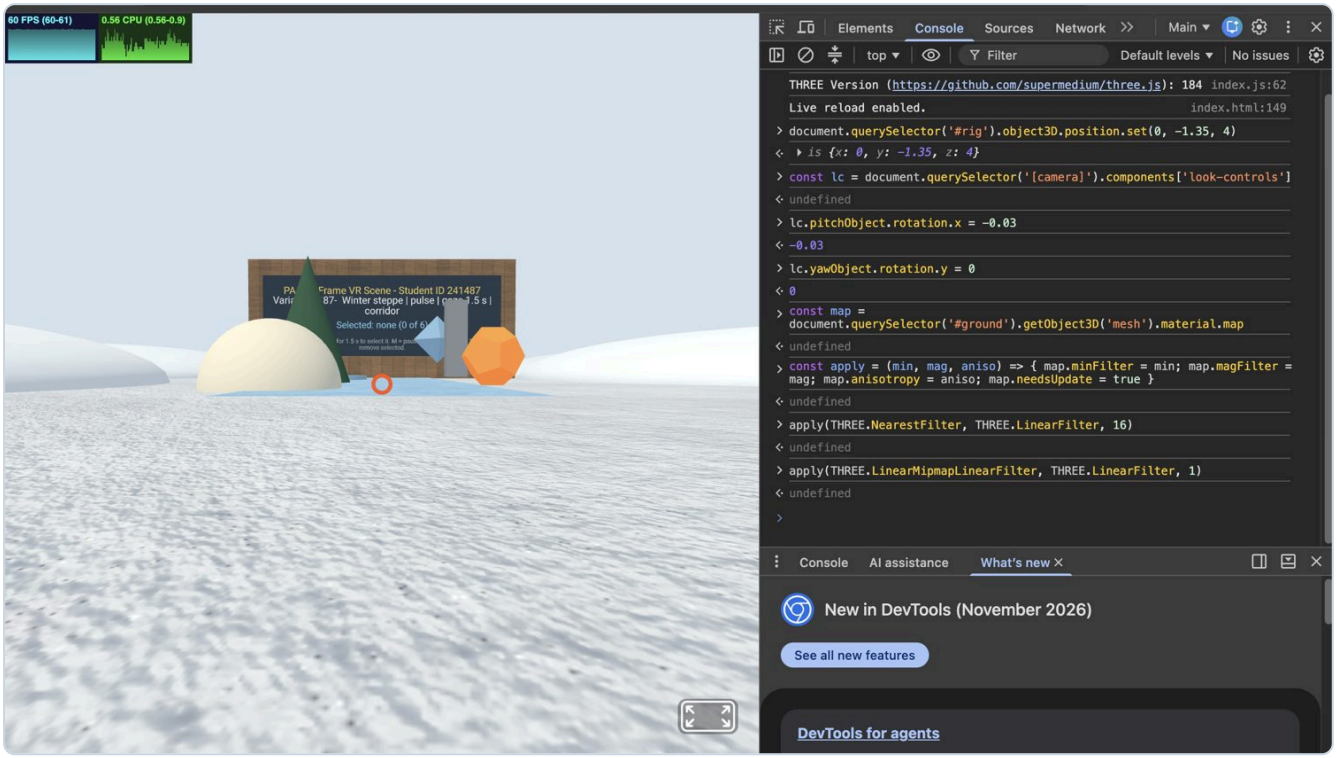


Figure 7. three.js defaults.

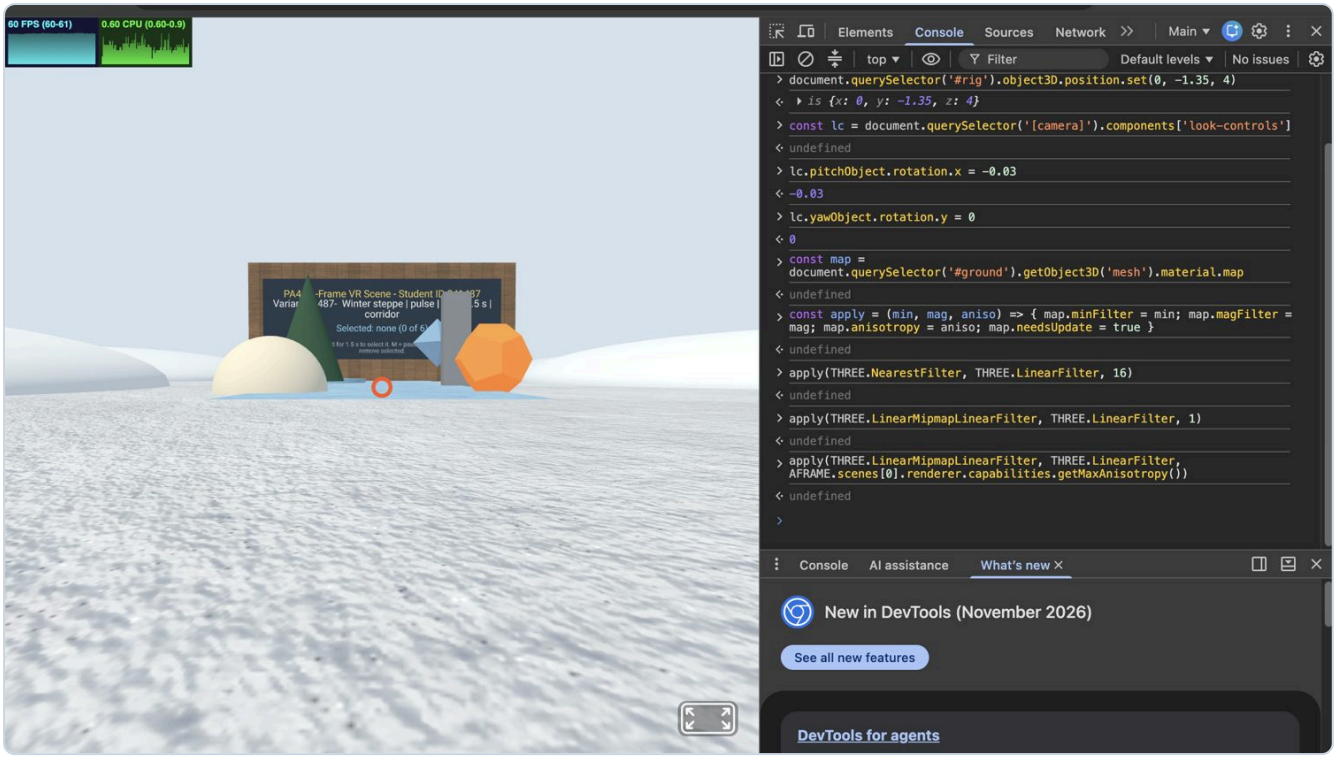


Figure 8. My filters, maximum anisotropy.

Explanation. Chapter 8 sets `minFilter = NearestFilter`, which turns mipmaps off and picks one texel per pixel, so distant snow flickers as the camera moves. `maxFilter` does not exist in three.js (the real name is `magFilter`), so that line does nothing.

Minification means many texels per pixel (far or grazing surfaces); *magnification* means one texel over many pixels (close up). *Mipmaps* are pre-filtered half-size copies; the GPU picks the level matching the distance. `LinearMipmapLinearFilter` blends two levels, removing shimmer; anisotropic filtering keeps detail on surfaces seen at low angles.

My choice: `LinearMipmapLinearFilter` for minification, `LinearFilter` for magnification, mipmaps on, and the maximum anisotropy the GPU supports.

E4 – What <a-scene> builds

Prediction. `<a-scene>` creates a three.js `WebGLRenderer`, a `Scene` and a `PerspectiveCamera`, runs the render loop, and starts the WebXR session when Enter VR is pressed.

Result. The console shows `sceneEl.renderer` with `isWebGLRenderer: true` (a three.js `WebGLRenderer`), `sceneEl.object3D.type` = `'Scene'` and `sceneEl.camera.type` = `'PerspectiveCamera'`. In `a-scene.js`, Enter VR calls `navigator.xr.requestSession(...)`, then `renderer.xr.setSession(...)` and stores `sceneEl.xrSession`.

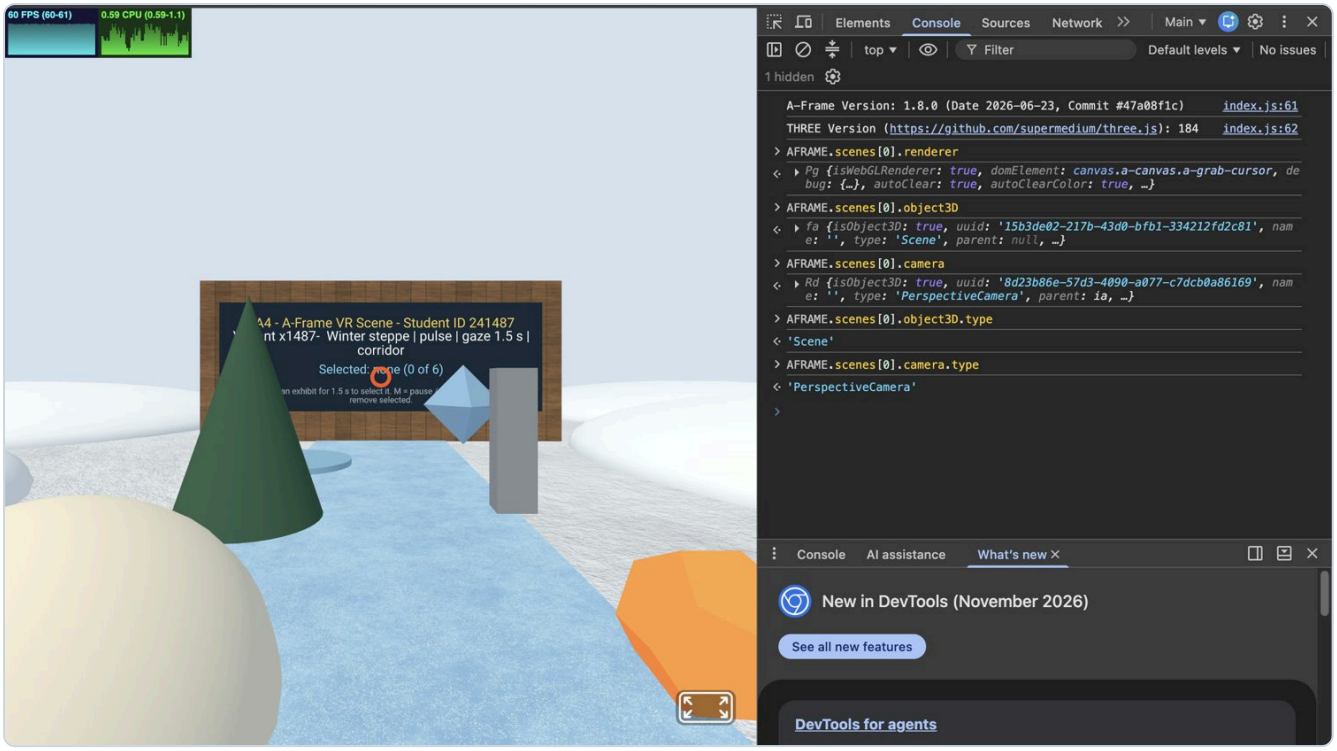


Figure 9. Renderer, scene, camera.

Explanation. A-Frame wraps what I wrote by hand in PA3: requesting an `immersive-ar` session with the `hit-test` feature through `navigator.xr.requestSession`, passing it to `renderer.xr.setSession`, and driving frames with my own `renderer.setAnimationLoop`. In A-Frame the scene element does all of this from HTML attributes.

Raw three.js is the better choice when a company needs full control of the render pipeline or custom shaders, for example a product configurator inside an existing web app. **A-Frame is the better choice** for fast prototypes built by small teams or designers, where declarative HTML saves weeks.

E5 – Performance

Prediction. About 20 draw calls; 60 more exhibits add about 60 (one per mesh) and may lower the fps; merging static meshes removes almost all of them.

Result. Readings from `renderer.info.render`, with fps from the `stats` panel:

Scene	fps	Draw calls	Triangles
Original, 6 exhibits	61	20	13,140
+ 60 exhibits added by script	60	80	86,582
60 static spheres merged into one mesh	60	21	86,580

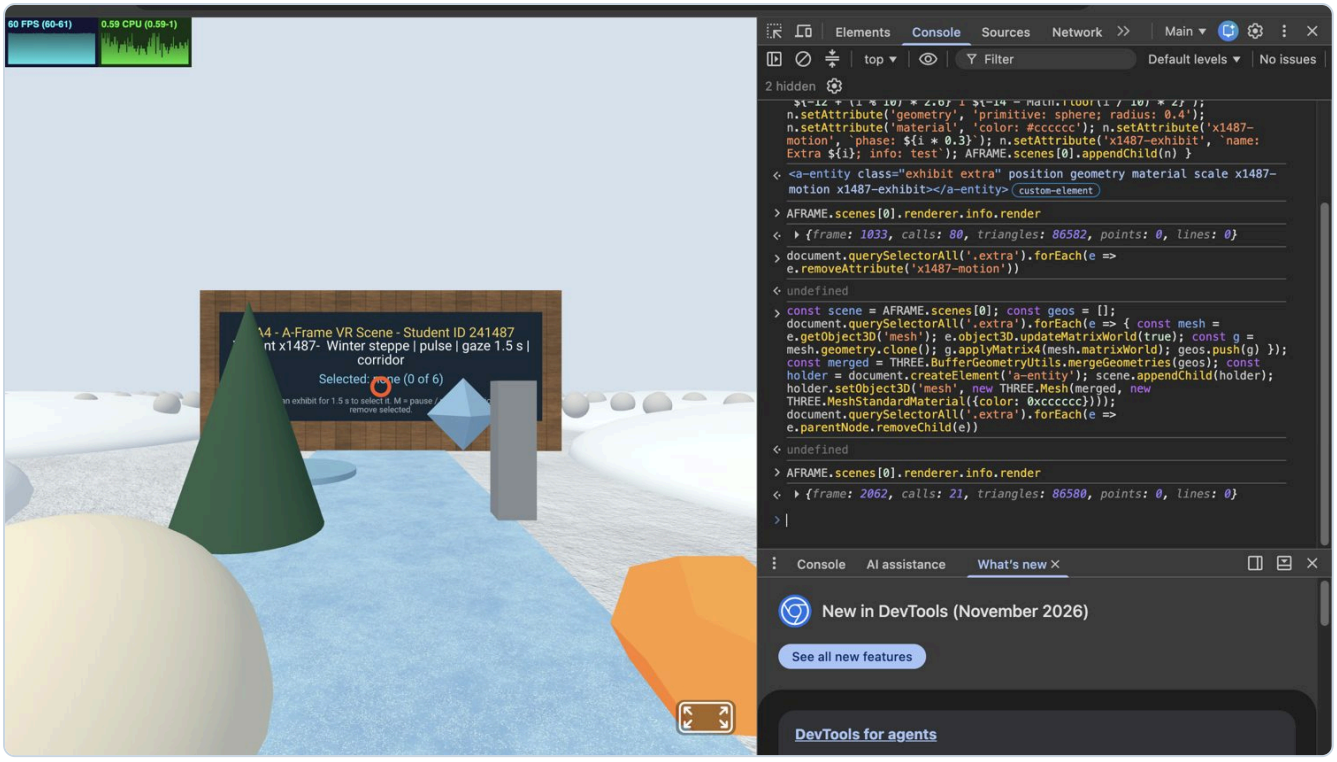


Figure 10. Draw-call readings.

Explanation. Each mesh with its own material costs one draw call, and each call costs CPU time even for few triangles. VR draws the scene once per eye at 72 to 90 fps on a mobile chip (11 to 14 ms per frame), so call overhead hurts far more than on a laptop.

Reduction: I stopped the 60 spheres' motion, baked their positions into their geometries and merged them with `THREE.BufferGeometryUtils.mergeGeometries` into one mesh. Draw calls fell from 80 to 21 with the same triangles (86,582 to 86,580); fps stayed at 60 on my laptop.

E6 – Comfort and design

Exhibits stand 2.5 to 8 m from the start and rows are 3 m apart, so each is at least 1.5 m from the walking line. Panel text is about 0.1 m high, so at 3 m it spans 2·atan(0.05/3) ≈ 1.9°, above the roughly 1.5° I treat as comfortable. The 1.5 s gaze fuse avoids accidental selections without tiring the neck, and the ring shrinks as feedback. This variant uses no lasers.

Comfort risk: smooth WASD walking in VR can cause motion sickness. I lowered the rig acceleration to 30, kept the corridor short and straight, and never rotate the view for the user; turning comes only from the head.

Development log

Two real problems from this project:

- Bug: I pasted several git commands at once, so `git init` ran in the parent folder instead of the PA4 folder and the first push failed.
- How I found it: the terminal showed an empty commit and `pwd` pointed to the wrong directory.
- Fix: I deleted the stray `.git`, ran `git init` inside `PA4_241487_Amangeldyyeva`, and made the first commit there, hash `714344f` on `main`.
- Second bug: the push failed with HTTP 400; setting `git config http.postBuffer 524288000` fixed it.

AI-usage note

This project was built with Claude (Anthropic, model Claude Sonnet 5.5) through Claude Code, with the instructor's permission to use AI in this course. Claude drafted the scene, `components.js`, the README, the experiment checklist and the first draft of this write-up. I ran every experiment in my own browser and took all screenshots, tested the scene in Chrome (normal and incognito windows) and in the Immersive Web Emulator, created the GitHub commits myself, and recorded the demo.